

Fizetések létehozása

1. Fizetesek.jsx

A fizetések kezeléséhez elkészítettem a Fizetesek.jsx komponenst. A komponensben React state-ek segítségével tároltam a fizetések, a szerződések és az űrlap adatait. Gondoskodtam arról, hogy az oldal betöltésekor automatikusan lekérdeződjenek a szükséges adatok a backendből. Az űrlap segítségével lehetővé tettem új fizetések rögzítését és a meglévő fizetések módosítását.

```
import { useEffect, useState } from "react";
import "../styles/fizetesek.css";

function Fizetesek() {

  const [fizetesek, setFizetesek] = useState([]);
  const [szerzodesek, setSzerzodesek] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujFizetes, setUjFizetes] = useState({
    szerzodesid: "",
    osszeg: "",
    fizetesdatum: "",
    statusz: "Teljesítve",
    fizetesmod: "",
    megjegyzes: ""
  });

  const fizetesekBetoltese = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/fizeteslistazas"
      );

      const adatok = await valasz.json();

      setFizetesek(adatok);

    } catch (err) {

      console.error(err);

    }

  };

}
```

Szakmai Gyakorlat II. – Frontend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
const szerzodesekBetoltese = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/szerzodeslistazas"
    );

    const adatok = await valasz.json();

    setSzerzodesek(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  fizetesekBetoltese();
  szerzodesekBetoltese();

}, []);

const mentes = async (e) => {

  e.preventDefault();

  try {

    let url =
      "http://localhost:3001/api/fizetesletrehozas";

    let method = "POST";

    if (szerkesztettId) {

      url =
        `http://localhost:3001/api/fizetesmodositas/${szerkesztettId}`;

      method = "PUT";

    }

    const valasz = await fetch(
```

```
        url,
        {
            method,
            headers: {
                "Content-Type":
                    "application/json"
            },
            body: JSON.stringify(
                ujFizetes
            )
        }
    );

    const adat =
        await valasz.json();

    alert(adat.uzenet);

    setUjFizetes({
        szerzodesid: "",
        osszeg: "",
        fizetesdatum: "",
        statusz: "Teljesítve",
        fizetesmod: "",
        megjegyzes: ""
    });

    setSzerkesztettId(null);

    fizetesekBetoltese();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

const torles = async (id) => {

    if (
        !window.confirm(
            "Biztosan törlöd a fizetést?"
        )
    ) {
```

Szakmai Gyakorlat II. – Frontend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/fizetestorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        fizetesekBetoltese();

    } catch (err) {

        console.error(err);

    }

};

return (

    <div className="tenant-container">

        <h1>Fizetések</h1>

        <form
            className="tenant-form"
            onSubmit={mentes}
        >

            <select
                value={
                    ujFizetes.szerzodesid
                }
                onChange={(e) =>
                    setUjFizetes({
                        ...ujFizetes,
                        szerzodesid:
                            e.target.value
                    })
                }
            />

        </form>

    </div>

);
```

```
    }
    required
  >

  <option value="">
    Válassz szerződést
  </option>

  {szerzodesek.map(
    (szerzodes) => (

      <option
        key={
          szerzodes.id
        }
        value={
          szerzodes.id
        }
      >
        {
          szerzodes.berlonev
        }
      </option>

    )
  )}

</select>

<input
  type="number"
  placeholder="Összeg"
  value={
    ujFizetes.osszeg
  }
  onChange={ (e) =>
    setUjFizetes({
      ...ujFizetes,
      osszeg:
        e.target.value
    })
  }
  required
/>

<input
  type="date"
  value={
```

```
        ujFizetes.fizetesdatum
      }
      onChange={ (e) =>
        setUjFizetes({
          ...ujFizetes,
          fizetesdatum:
            e.target.value
        })
      }
      required
    />

    <select
      value={
        ujFizetes.statusz
      }
      onChange={ (e) =>
        setUjFizetes({
          ...ujFizetes,
          statusz:
            e.target.value
        })
      }
    >

      <option value="Teljesítve">
        Teljesítve
      </option>

      <option value="Függőben">
        Függőben
      </option>

      <option value="Sikertelen">
        Sikertelen
      </option>

    </select>

    <input
      type="text"
      placeholder="Fizetési mód"
      value={
        ujFizetes.fizetesmod
      }
      onChange={ (e) =>
        setUjFizetes({
          ...ujFizetes,
```

```
                fizetesmod:
                    e.target.value
            })
        }
        required
    />

    <input
        type="text"
        placeholder="Megjegyzés"
        value={
            ujFizetes.megjegyzes
        }
        onChange={ (e) =>
            setUjFizetes({
                ...ujFizetes,
                megjegyzes:
                    e.target.value
            })
        }
    />

    <button
        type="submit"
    >
        {
            szerkesztettId
                ? "Módosítás mentése"
                : "Fizetés rögzítése"
        }
    </button>

</form>

<div className="filter-container">

    <select
        value={statuszSzuro}
        onChange={ (e) =>
            setStatuszSzuro(
                e.target.value
            )
        }
    >

        <option value="Mind">
            Összes fizetés
        </option>
```

```
        <option value="Teljesítve">
            Teljesítve
        </option>

        <option value="Függőben">
            Függőben
        </option>

        <option value="Sikertelen">
            Sikertelen
        </option>

    </select>

</div>

<div className="tenant-grid">

    {fizetesek
        .filter((fizetes) => {

            if (
                statuszSzuro ===
                "Mind"
            ) {
                return true;
            }

            return (
                fizetes.statusz ===
                statuszSzuro
            );

        })
        .map(
            (fizetes) => (

                <div
                    className="tenant-card"
                    key={
                        fizetes.id
                    }
                >

                    <h3>
                        {
                            fizetes.berlonev
```



```
    }
</h3>

<p>
    <strong>
        Összeg:
    </strong>{" "}
    {
        fizetes.osszeg
    } Ft
</p>

<p>
    <strong>
        Dátum:
    </strong>{" "}
    {
        new Date(
            fizetes.fizetesdatum
        ).toLocaleDateString(
            "hu-HU"
        )
    }
</p>

<p>
    <strong>
        Fizetési mód:
    </strong>{" "}
    {
        fizetes.fizetesmod
    }
</p>

{
    fizetes.megjegyzes &&
    (
        <p>
            <strong>
                Megjegyzés:
            </strong>{" "}
            {
                fizetes.megjegyzes
            }
        </p>
    )
}
```

```
<p>

    <strong>
        Státusz:
    </strong>

    <span
        className={
            fizetes.statusz === "Teljesítve"
            ? "active-status"
            : fizetes.statusz === "Függőben"
            ? "pending-status"
            : "expired-status"
        }
    >
        {" "}
        {
            fizetes.statusz
        }
    </span>

</p>

<div className="buttons">

    <button
        type="button"
        className="edit-btn"
        onClick={() => {

            setSzerkesztettId(
                fizetes.id
            );

            setUjFizetes({
                szerzodesid:
                    fizetes.szerzodesid,
                osszeg:
                    fizetes.osszeg,
                fizetesdatum:
                    fizetes.fizetesdatum?.split("T")[0] ||
                    fizetes.fizetesdatum,
                statusz:
                    fizetes.statusz,
                fizetesmod:
                    fizetes.fizetesmod,
                megjegyzes:
                    fizetes.megjegyzes || ""
            })
        }}
    >
        Szerkesztés
    </button>

    <button
        type="button"
        className="delete-btn"
        onClick={() => {
            deleteFizetes(fizetes.id)
        }}
    >
        Töröl
    </button>

</div>
```

Szakmai Gyakorlat II. – Frontend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        });

        window.scrollTo({
            top: 0,
            behavior: "smooth"
        });

    }}
    >
    Módosítás
</button>

<button
  type="button"
  className="delete-btn"
  onClick={() =>
    torles(
      fizetes.id
    )
  }
>
  Törlés
</button>

</div>

</div>

)
}}

</div>

</div>

);

}

export default Fizetesek;
```

Szakmai Gyakorlat II. – Frontend fejlesztési szakasz

Orosz Kristóf – EYZWG9

localhost:3000/fizetesek

Fizetések

Válassz szerződést ▾ Osszeg éééé . hh . nn . Teljesítve ▾ Fizetési mód

Megjegyzés **Fizetés rögzítése**

Összes fizetés ▾

Biró Zoltán

Összeg: 14500.00 Ft

Dátum: 2026. 06. 19.

Fizetési mód: Bankkártya

Megjegyzés: Új fizetési kötelezettség!

Státusz: Függőben

Módosítás **Törítés**

1.Kép: Fizetések HTML oldala a funkciókkal ellátva.